Lab11–DataStructureswithAI: ImplementingFundamental Structures

2303A51939

Batch-27

Task1–StackImplementation

Code:

```
classStack:
    def __init__(self):
        self.items = []
    defpush(self,item):
        self.items.append(item)
    def pop(self):
        if not self.is_empty():
            returnself.items.pop()
        return"Stackisempty"
    def peek(self):
        if not self.is_empty():
            returnself.items[-1]
        return"Stackisempty"
```

```
    defis_empty(self):
        returnlen(self.items)==0
```

Task2–QueueImplementation Code:

```
classQueue:
    def __init__(self):
        self.items = []
    def enqueue(self, item):
        self.items.append(item)
    defdequeue(self):
        if self.items:
            returnself.items.pop(0)
        return "Queue is empty"
    defpeek(self):
        ifself.items:
            return self.items[0]
        return"Queueisempty"
    defsize(self):
        returnlen(self.items)
```

Task3–SinglyLinkedList Code:

```
class Node:
    def __init__(self,data):
        self.data = data
        self.next = None
class LinkedList:
    def __init__(self):
        self.head=None
    def insert(self, data):
        new_node=Node(data)
        if not self.head:
            self.head=new_node
            return
        temp=self.head
        whiletemp.next:
            temp=temp.next
        temp.next=new_node

    def display(self):
        temp=self.head
        while temp:
```

```
        print(temp.data,end="->")
        temp = temp.next
    print("None")
```

Task4–BinarySearchTree(BST) Code:

```
classBST:
  def __init__(self,key):
    self.key = key
    self.left = None
     self.right=None
  definsert(self,value): if
     value < self.key:
        if self.left is None:
            self.left=BST(value)
        else:
            self.left.insert(value)
    else:
        if self.right is None:
            self.right=BST(value)
        else:
            self.right.insert(value)
```

```
definorder(self):
    if self.left:
        self.left.inorder()
    print(self.key,end="") if
    self.right:
        self.right.inorder()
```

Task5–HashTable(Chaining)

Code:

```
class HashTable:
    def __init__(self,size=10):
        self.size = size
        self.table=[[]for_inrange(size)]
    def _hash(self, key):
        returnhash(key)%self.size def
    insert(self, key, value):
        index=self._hash(key)
        self.table[index].append((key,value))
    def search(self, key):
        index=self._hash(key)
        fork,vinself.table[index]: if
            k == key:
                returnv
```

```
        return"Keynotfound"
    def delete(self, key):
        index=self._hash(key)
        fori,(k,v)inenumerate(self.table[index]): if
            k == key:
                delself.table[index][i]
                return "Deleted"
        return"Keynotfound"
```

Task6–Graph(AdjacencyList)

Code:

```
classGraph:
    def __init__(self):
        self.graph = {}
    defadd_vertex(self,vertex):
        ifvertexnotinself.graph:
            self.graph[vertex]=[]
    defadd_edge(self,v1,v2):
        self.add_vertex(v1)
        self.add_vertex(v2)
        self.graph[v1].append(v2)
        self.graph[v2].append(v1)
```

```
    defdisplay(self):
        forvertexinself.graph:
            print(vertex,"->",self.graph[vertex])
```

Task7–PriorityQueue

Code:

```
importheapq
classPriorityQueue:
    def __init__(self):
        self.heap=[]
    def enqueue(self, priority, item):
        heapq.heappush(self.heap,(priority,item))
    defdequeue(self):
        if self.heap:
            returnheapq.heappop(self.heap)
        return "Queue is empty"
    def display(self):
        print(self.heap)
```

Task8–Deque

Code:

```
fromcollectionsimportdeque
```

```
class DequeDS:
    def __init__(self):
        self.deque=deque()
    def insert_front(self, item):
        self.deque.appendleft(item)
    def insert_rear(self, item):
        self.deque.append(item)
    def remove_front(self):
        returnself.deque.popleft()
    def remove_rear(self):
        returnself.deque.pop()

)
```

Task9–CampusResourceManagementSystem Feature

→ Data Structure Mapping

Feature	**Data Structure**	**Justification**
StudentAttendance Tracking	HashTable	FastlookupbystudentID.O(1)average time complexity.
EventRegistration	LinkedList	Dynamicinsertionandremovalof participants.
LibraryBookBorrowing	BST	BookscanbeorganizedbyIDforsorted access.

Feature	Data Structure	Justification
BusScheduling	Graph	Routesandstopsformconnected networks.
CafeteriaOrderQueue	Queue	OrdersmustbeservedinFIFOorder.

ImplementedFeature:CafeteriaOrderQueue

```
classCafeteriaQueue:
    """ManagestudentfoodordersusingQueue."""

    def __init__(self):
        self.orders = []

    def place_order(self, student_name):
        self.orders.append(student_name)
        print(f"{student_name}placedorder.")

    defserve_order(self):
        if self.orders:
            served=self.orders.pop(0)
            print(f"Serving {served}")
        else:
            print("Noorderstoserve.")

    def display_orders(self):
        print("CurrentOrders:",self.orders)


#Example
cq=CafeteriaQueue()
```

```
cq.place_order("Rahul")
cq.place_order("Anita")
cq.serve_order()
cq.display_orders()
```

Task10–SmartE-CommercePlatform

Feature → Data Structure Mapping

Feature	**Data Structure**	**Justification**
Shopping Cart	Linked List	Dynamicaddition/removalofproducts.
Order Processing	Queue	Orders processed in FIFO order.
Top-SellingProducts	Priority Queue	Rankedbyhighestsales.
ProductSearch	Hash Table	FastproductlookupbyID.
DeliveryRoute Planning	Graph	Warehousesandlocationsformnetwork paths.

ImplementedFeature:ProductSearch(HashTable)

```
classProductSearch:
    """ProductlookupsystemusingHashTable."""

    def __init__(self):
        self.products={}

    defadd_product(self,product_id,name):
        self.products[product_id] = name
```

```
    defsearch_product(self,product_id):
        returnself.products.get(product_id,"Productnotfound")


#Example
ps = ProductSearch()
ps.add_product(101,"Laptop")
ps.add_product(102,"Mobile")

print(ps.search_product(101))
print(ps.search_product(200))
```